

Contents

Anforderungen	1
Rahmen	1
Funktionale Anforderungen	1
Technische Anforderungen	1
Prozess-Anforderungen (Scrum)	1
Liefergegenstände (Auftrag an die Umsetzung)	2
Im Planungsgespräch geklärte Design-Entscheidungen	2
Bewusst gewählte Vereinfachung (kein offener Punkt, aber erwähnenswert)	2

Anforderungen

Ausgangslage und Vorgaben, aus denen dieser Kurs entstanden ist. Dient als Referenz, falls der Kurs später angepasst oder auf andere Rahmenbedingungen übertragen wird.

Rahmen

- Schülerpraktikum, Gymnasium, 10. Klasse, 7 Schüler in 3 Teams.
- Vorlauf bereits absolviert (nicht Teil dieses Repos): 2 Tage Kotlin-Grundlagen inkl. Objektorientierung, IntelliJ-Setup, Konsolen-I/O, Einführung Scrum (Pair Programming, Unit-Tests).
- Fortsetzung: 3 weitere Tage. Tag 1 kurz (9:00–13:00, danach externer KI-Vortrag), Tag 2+3 jeweils 9:00–15:00 Uhr.
- Ziel der 3 Tage: tiefer in Kotlin eintauchen, echte Logik programmieren statt nur Konsolen-Übungen.

Funktionale Anforderungen

- Kotlin-Programm, in dem Roboter gegeneinander antreten, auf einem 10×10-Raster.
- Robot als Interface-Konzept; Schüler schreiben Logiken wie `makeMove`, `makeAttack` (später konkretisiert zu einer einzigen `decide(sensors): Action`-Entscheidungsfunktion).
- Sensoren, mit denen ein Bot seine Umgebung/Gegner wahrnehmen kann.
- Schüler programmieren echte Verhaltensregeln, Beispiele aus dem Auftrag:
 - „Wenn Leben < 20, dann flüchte“
 - „Wenn Gegner im Visier, dann Dauerfeuer“
- Testbetrieb: Teams lassen ihre Bots gegeneinander laufen, um Logik zu prüfen.
- Abschluss an Tag 3: alle drei Gruppen lassen ihre Roboter gegeneinander antreten (Turnier/Finale).

Technische Anforderungen

- Sprache: Kotlin.
- UI: Compose for Desktop (Teil von Compose Multiplatform).
- Da in 3 Tagen für Kotlin-Anfänger nicht alles neu erstellbar ist: ein fertiges Framework muss vorliegen, das insbesondere das Zeichnen/Rendering übernimmt.
- Schüler bekommen einzelne Aufgaben, um das Framework zu erweitern bzw. Roboter-Logik zu programmieren — sie schreiben keinen UI-/Engine-Code.
- Gerüst muss vollständig fertig sein, inklusive vorgefertigter Lösungen zu den Aufgaben für den Dozenten.

Prozess-Anforderungen (Scrum)

- Entwicklung im Scrum-Prozess.

- Backlog-Items für die Schüler, aber eigene Kreativität soll möglich sein (Kür-Aufgaben).
- Tägliches Daily.
- Ein Scrum-Board.
- Abschließendes Review.
- Unit-Tests optional, nicht verpflichtend.

Liefergegenstände (Auftrag an die Umsetzung)

- Vollständiger Kurs für die drei Tage, als Markdown-Dateien, mit Programm für jeden Tag.
- Fertiges Kotlin-Framework (Zeichnen, Engine) als Basis zum Aufbauen.
- Backlog-Items inklusive Lösungen für den Dozenten.
- CLAUDE.md für technische Weiterarbeit am Code.
- README.md: wie man startet, sich einarbeitet usw.
- Detaillierter Zeitplan für alle drei Tage.
- Sinnvolle Ordnerstruktur für alle Artefakte.
- Alles, was der Dozent wissen muss, in Markdown-Dateien dokumentiert.
- Anwendung zum Testen tatsächlich starten (nicht nur Code liefern, sondern auch verifizieren).

Im Planungsgespräch geklärte Design-Entscheidungen

Diese Punkte waren im ursprünglichen Auftrag offen und wurden vor der Umsetzung festgelegt:

Frage	Entscheidung
Kampf-Modus	Tick-basiert, Echtzeit-Optik (kein striktes Rundensystem)
Bot-API-Stil	Reine Entscheidungsfunktion <code>decide(sensors): Action</code> , kein eigener Kontroll-Loop im Schülercode
Netzwerk-Umgebung	Internet beim Gradle-Build verfügbar (Standard-Firmennetz), kein Offline-Cache vorbereitet
Scrum-Board	Physisch (Whiteboard/Post-its), ein gemeinsames Board für alle drei Gruppen (kein Board pro Team)
Projekt-/Modul-Struktur	Ein Gradle-Projekt, drei Bot-Packages (bots/teama, bots/teamb, bots/teamc), keine separaten Gradle-Module
Versionskontrolle	Kein Git — loser Ordner. Integration der Team-Bots erfolgt manuell auf dem Dozenten-Rechner (später revidiert : tatsächlich umgesetzt wurde ein Git-Branch/PR-Workflow, siehe setup.md und ../git-github-basics.md)

Bewusst gewählte Vereinfachung (kein offener Punkt, aber erwähnenswert)

Die Engine kennt kein Freund-Feind-Konzept: `RobotState.teamName` wird 1:1 aus `RobotBrain.name` übernommen, es gibt keine Gruppierung mehrerer Bot-Instanzen zu einem gemeinsam kämpfenden Team. Jeder Bot tritt einzeln gegen jeden anderen an (reines Free-for-all). Die Team-Zuordnung in `BotRegistry.allTeams` ist rein organisatorisch (Anzeige/Auswahl), ohne Auswirkung auf die Spielregeln. Diese Vereinfachung wurde nicht explizit im Auftrag gefordert, aber nicht widersprochen und aus Zeit-/Komplexitätsgründen für ein 3-Tage-Praktikum so umgesetzt.